College of Engineering and Information Technology

Flying Radar: A Drone-Based Software-Defined Radar

Commercial In Confidence

ENGI 4900: Final Report

13 Nov 2026

36/36 Pages

Summary of AI Use

Categories of artificial intelligence (AI) tool use in this assignment are as indicated in the following table.

AI tool use category	**Used**
Spelling	✓
Grammar	✓
Other copy editing	
Literature review support	
Visuals	
Substantive editing	
Idea generation	
Translation	
Research technical support	✓
No AI tool was used	

Acknowledgements

Contents

Summary of AI Use i

Acknowledgements ii

List of Figures v

List of Tables vi

List of Acronyms viii

Contribution Statement ix

4 Methods, Results and Discussion xi

5 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements xii

5.1 Introduction . . . xii

5.2 Methodology . . . xii

5.2.1 Hardware and Setup . . . xii

5.2.2 Development Approach . . . xiii

5.2.3 Flight Controller Configuration . . . xv

5.2.4 Final System . . . xv

5.2.5 Post-Processing . . . xvii

5.3 Results and Discussion . . . xviii

5.3.1 GPS Drift . . . xviii

5.3.2 RC Switch Trigger Latency . . . xix

5.3.3 Computational Overhead . . . xx

5.4 Summary . . . xx

References xxi

Statement of AI Use xxii

6 System Design xxiii

6.1 Problem Definitions . . . xxiii

6.2 Plan for Work . . . xxiv

6.2.1 Objective 1: Develop SDR-to-SBC Interface for Radar Control and Data Acquisition . . . xxiv

6.2.2 Objective 2: Design and Integrate a Mechanical Mounting and Protection System for the Payload . . . xxv

6.2.3 Objective 3: Design, Build, and Verify the Onboard Power System xxvii
6.2.4 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements xxviii

A Objective 4: Supporting Material xxx

A.1 SSH Setup xxx
A.2 GPIO Packages Install xxx
A.3 Virtual Environment and `rpi-lgpio` Install xxx
A.4 GPIO Read Script xxx
A.5 Telemetry Data Visualisation Script xxxi

List of Figures

5.1 Activation signal flow starting at the pilot transmitter and ending at the software-defined radar (SDR). xii

5.2 Control system development cycle xiv

5.3 Captured screenshot of live data display for the MAVLink interface (Note: data has demo values). xvii

5.4 State diagram of radar control sequence xvii

5.5 Flight path and radar capture locations plotted in the data visualisation script. . xviii

List of Tables

5.1 MAVLink messages requested from the flight controller and their use. xv
5.2 Mean position of each stationary window xix
5.3 Horizontal GPS drift and satellites visible for each stationary window xix
5.4 Trigger-to-capture latency after removing the 4 s settle-delay from logs 1–4 . . . xix
5.5 Raspberry Pi resource usage for each test condition xx

6.1 problems, constraints and the objective addressing each xxiv

List of Acronyms

Acronym	Definition
3D	three dimensional
AC	alternating current
AI	artificial intelligence
API	application programming interface
ASA	acrylonitrile styrene acrylate
AST	Advanced Systems & Technologies
CLI	command-line interface
CPU	central processing unit
C-UAS	counter-uncrewed aerial system
CAD	computer-aided design
COTS	commercial off-the-shelf
CW	continuous wave
DC	direct current
DEM	digital elevation model
FMCW	frequency modulated continuous wave
GPIO	general purpose input/output
GPR	ground penetrating radar
GPS	global positioning system
HALE	high-altitude long-endurance
IDE	integrated development environment
IED	improvised explosive device
InSAR	interferometric synthetic aperture radar
IQ	in-phase and quadrature
JSON	JavaScript Object Notation

JST	Japan solderless terminal
LiPo	lithium-polymer
PC	personal computer
PCB	printed circuit board
PID	product identifier
PLA	polylactic acid
PWM	pulse-width modulation
RAM	random-access memory
RC	radio control
RF	radio frequency
SAR	synthetic aperture radar
SBC	single-board computer
SD	secure digital
SDR	software-defined radar
SI	International System of Units
SLD	single-line diagram
SSH	secure shell
UAV	uncrewed aerial vehicle
USB	universal serial bus
VID	vendor identifier

Contribution Statement

This contribution statement records the specific contribution of each team member to the technical work and preparation of this progress report. Contributions are grouped according to the simplified CRediT categories of Approach, Technical output, Writing, and Project management.

Table CS1: Declaration of group contributions to the report.

Author	Team Role	Approach	Technical Output	Writing	Project Management

Table CS2: Acknowledgement of AI Tool Use

Group member name	**Specific sub-sections or paragraphs where AI was used**	**AI Tool Use Category/ies**

Table CS3: Group Member Comments

Group member name	**Comments**

By signing below students are acknowledging that the contributions recorded above are a true representation of the contribution made by each individual.

Table CS4: Signature Acknowledgement of Contribution

Name	**Student ID**	**Signature**

4 Methods, Results and Discussion

5 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements

This chapter provides a detailed account of the fourth objective of the project. The objective and context is introduced, followed by the methodology explained in detail that was followed to complete the objective. Finally, the results of the subsequent verification of the objective completion are discussed.

5.1 Introduction

To synchronise flight telemetry and radar measurements, a pilot-initiated control signal must be interpreted, processed, and transmitted through the system. The signal travels from the pilot transmitter to the receiver, then to the flight controller, and finally to the single-board computer (SBC), which triggers SDR logging. This signal path is shown in Figure 5.1. Synchronising the radar captures and the telemetry is critical, as each radar capture needs a position and attitude before it can be georeferenced or used in later processing.

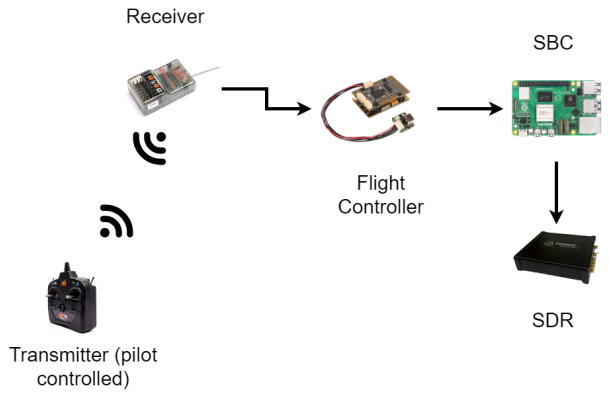


Figure 5.1: Activation signal flow starting at the pilot transmitter and ending at the SDR.

The work done in this objective included researching and procuring a suitable SBC, configuring the flight controller, designing software to interface with the flight controller and radar, and implementing a post-processing script. The requirements of the system was the ability for a pilot to trigger the radar capture directly from a switch on the transmitter, and that every capture is synchronised to the telemetry for processing. The final system was a MAVLink-based Python program on board a Raspberry Pi 5 that interfaced with both the radar and the flight controller.

5.2 Methodology

The methodology provides a detailed account of the process underwent to complete the fourth objective. This includes the procurement of the SBC and setup. Subsequently, the approach to the program development on the SBC is outlined. The configuring of the flight controller is also accounted. The final system is then explained in detail including architecture and design choices.

5.2.1 Hardware and Setup

First, requirements of the SBC were identified. The hardware requirements for the SBC were derived from the SDR datasheet, which specified the need for a high-speed USB 2.0 interface

(480 Mb/s) [?]. Additionally, the SBC should provide sufficient processing capability to support potential future expansion of onboard processing capability, placing requirements on central processing unit (CPU) performance and random-access memory (RAM) capacity. Consequently, a Raspberry Pi 5 with 16 GB of RAM was selected and procured. In addition to satisfying the hardware requirements, Raspberry Pi devices were selected because of their well-supported software ecosystem and prior use in drone-mounted radar research applications [?, ?, ?, ?]. A protective case and active cooler were also acquired to secure the device on board the drone and manage CPU temperature during SDR operation.

The Raspberry Pi 5 was physically assembled and configured with a lightweight Ubuntu 22.04 operating system. A remote development workflow was then configured to develop and test the Python script used to read the general purpose input/output (GPIO) pin. Native development on the Raspberry Pi was avoided due to its lower performance compared with a desktop computer. Instead, secure shell (SSH) was configured and integrated with an integrated development environment (IDE) to enable remote development. The setup commands are shown in Appendix A.1. To allow the Raspberry Pi to read GPIO pins, the required GPIO packages were installed. The setup commands are shown in Appendix A.2. One of the packages required a Python virtual environment. The setup and installation of this package and a Python virtual environment is shown in Appendix A.3. After installing the packages, a Python script was created to read a GPIO pin and respond when an external component pulls the signal high. The script is shown in Appendix A.4. To verify the program, an Arduino Uno R3 was used to generate an input signal for a GPIO pin. After the setup was confirmed working, the objective moved toward the development stage.

5.2.2 Development Approach

The development approach followed the diagram shown in Figure 5.2. Section 5.2.1 detailed the SBC setup and the GPIO read. The next development of the objective read the flight controller's PWM signal directly from the GPIO pins, afterward this was revised for MAVLink to be used as the interfacing protocol in place of the GPIO pins.

The flight controller uses pulse-width modulation (PWM) as the method to encode data to be sent. The protocol encodes data in the pulse width of square wave signals and is typically used for servo control [?]. Whether a signal was logic HIGH or logic LOW was determined by the pulse width of the transmission. For the flight controller signal, a pulse width below 1300 μs was logic LOW, and a pulse width above 1700 μs was logic HIGH. The gap between LOW and HIGH acts as a dead-band for noisy signals.

Initially, the PWM signal was read directly from servo output pins into the GPIO pins on the SBC. This was eventually replaced with a direct USB connection using the MAVLink protocol to read in the radio control (RC) channel signal directly from the flight controller communications, instead of reading a servo output. Doing this simplified physical wiring greatly as with MAVLink all communication is handled over a single universal serial bus (USB) connection. The GPIO activation signal was also dependent on the safety switch status of the drone; it had to be off for an output to occur. This issue is not present in the MAVLink setup.

After the MAVLink software was created, a separate post-processing script in Python to be used on a desktop PC to process the data was developed. This concluded the development cycle of the control system.

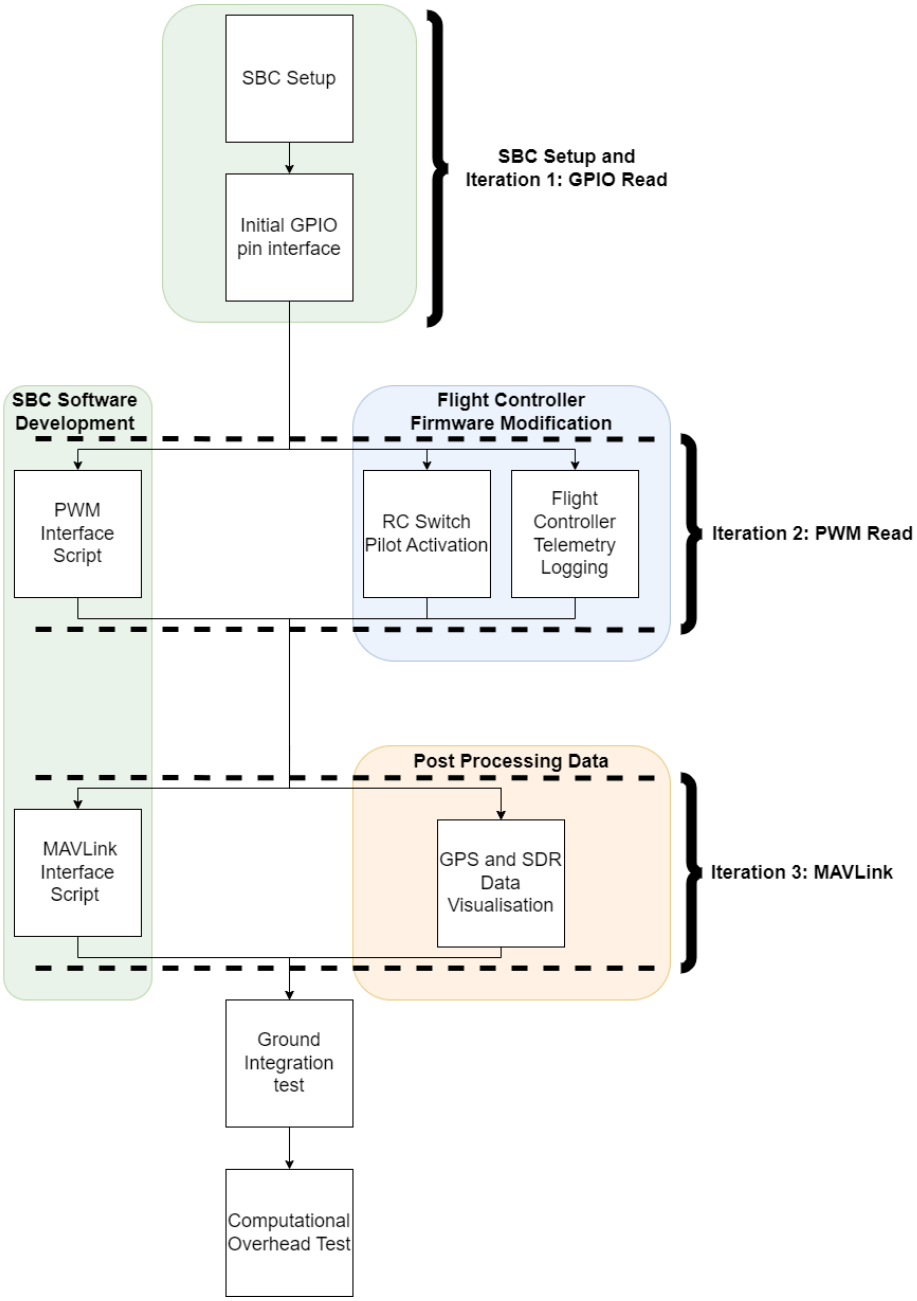


Figure 5.2: Control system development cycle

5.2.3 Flight Controller Configuration

The flight controller needed to be configured to read in a signal from the transmitter and tie that signal to a specific channel. On the transmitter in possession, there were 8 physical switches. The switch that was deemed suitable was tied to RC channel 15 using the transmitter settings. RC Channel 15 was then tied to servo output 8. After configuring, the output signal was scoped with an oscilloscope and confirmed working as intended. After the system swapped to using MAVLink, the servo output was no longer used and PWM was instead read from the MAVLink connection.

It was also found that when the host PC connects to the flight controller, the flight controller makes a .tlog file and continues to write to the .tlog file until the connection is terminated. The telemetry log only records global positioning system (GPS) when the drone is armed. This setting did not suit the project, as the drone needed to be disarmed for the majority of the project. To enable GPS logging while disarmed, a parameter was changed in the config file on the flight controller. Eventually the .tlog file was replaced by a .csv file on board the SBC as it was able to read all relevant telemetry status' directly.

5.2.4 Final System

A single python 3 program on the SBC handled both the flight controller and the SDR in one loop. The code was split by device, a `Drone` class, which handled the flight controller interface, a `Radar` class, which handled the radar interface, and a small `main.py` script to run the overall program.

The benefit to the program being one system was the fact that all the timing was able to use a single clock, which was `runtime` (seconds elapsed since the start of the program). Both the telemetry log and the radar used this value, and shared the same time base which is what made synchronisation possible.

Mavlink Interface

The SBC interfaced with the drone via Mavlink, a communication protocol used to interface between flight controllers and desktop computers, which the Raspberry Pi 5 running Ubuntu resembles in architecture. The physical connection was over USB, and the software interface was handled by the `pymavlink` python package.

The script blocked until it received a `HEARTBEAT` signal which confirmed the link and gave the target system and component IDs to be used by later commands. Instead of relying on default stream rates, the script sent `MAV_CMD_SET_MESSAGE_INTERVAL` commands to request each message at a fixed rate. The message IDs of the key messages is shown in Table 5.1.

Table 5.1: MAVLink messages requested from the flight controller and their use.

Message	ID	Rate	Used for
`GPS_RAW_INT`	24	1 Hz	Fix type, satellites, latitude, longitude
`GLOBAL_POSITION_INT`	33	5 Hz	Altitude (AMSL and relative), groundspeed
`ATTITUDE`	30	5 Hz	Roll, pitch, yaw
`BATTERY_STATUS`	147	1 Hz	Voltage, current, remaining capacity
`VFR_HUD`	74	2 Hz	Requested but not parsed
`RC_CHANNELS`	65	5 Hz	Channel 15 (radar trigger)
`HEARTBEAT`	0	Default	Armed state, flight mode

On each loop in the `main.py` file, `update()` drained every queued message with non-blocking

`recv_match(blocking=False)`, and so the program was never stalled by a slow link as the stored value was always the latest received. Raw Mavlink fields were converted to International System of Units (SI) units upon reception, including converting latitude and longitude from degE7 to degrees, altitude from millimetres to metres, and voltage from millivolts to volts. The armed status of the drone was read from the `MAV_MODE_FLAG_SAFETY_ARMED` bit of `HEARTBEAT.base_mode`, and the flight mode was decoded with `mode_string_v10`.

The radar was triggered from a switch on the pilot's transmitter which was mapped to channel 15. However, instead of reading in a PWM signal output from the flight controller, the signal was read through the `RC_CHANNELS` MAVlink message. The trigger logic remained the same; rising edge detection. A capture was requested by the main script to the radar only when the pulse width was above 1800 μs. The capture logic was sticky; the switch had to be seen as low before a new capture could be requested (which was a pulse width below 1200 μs). The deadband was widened from the previous iteration as it was found via oscilloscope that a valid signal never goes above 1200 μs or below 1800 μs. The sticky logic prevented one switch from firing repeatedly, ignored the noise in the invalid range, and prevented a switch left high at startup from requesting a capture. Both pulse width parameters can be changed easily in the config file. Buffering was not used; if the radar was still busy, the request was rejected and a message printed, instead of the request being queued. The reason for this was predictability; the timespan of the radar capture is too long for buffering to be feasible.

On each script startup a telemetry log in the form of a .csv file was created from the program, named with the start date and time, so that a telemetry log never overwrote a previous log. Each row in the log held sixteen telemetry fields and five radar fields of data, in addition to the program runtime. Among the five radar fields was the file name of the radar capture (if one occurred), so that each radar capture could be matched directly to the telemetry rows.

To view live data for verification and testing, a terminal dashboard (which was viewed over SSH) showed flight state, GPS, attitude, battery voltage, the pulse width of the RC channel tied to the activation of the SDR, and the radar status. The display was refreshed at one Hz; a parameter changeable in the config file. A screenshot of the display with demo values is shown in Figure 5.3.

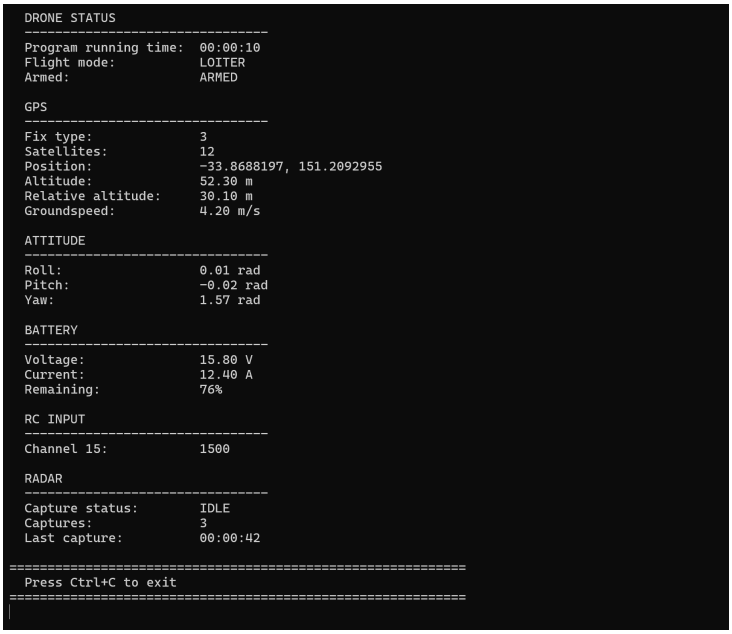


Figure 5.3: Captured screenshot of live data display for the MAVLink interface (Note: data has demo values).

SDR Interface

The SBC and radar did not communicate directly. Instead, the capture executable file was run by the `Radar` class in the program as a subprocess. This was to ensure appropriate separation between the two systems in order to minimise complexity. Under this method, the SBC was only responsible for scheduling and bookkeeping of the radar captures.

The capture settings for the radar existed in a JavaScript Object Notation (JSON) file which was read by the program at startup, so that parameters could change between flights without editing code. Among these settings were the executable and firmware paths, output folder for the radar captures, duration of capture, and sweep time. The radar capture cycle was modelled as a state machine, the diagram is shown in Figure **??**.

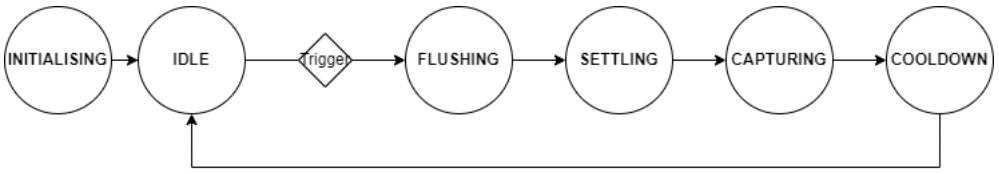


Figure 5.4: State diagram of radar control sequence

Each capture file was named according to its capture number and runtime on capture. Together with the per-row radar columns in the CSV file, this gave two independent ways to align the radar capture data with the telemetry data sets.

5.2.5 Post-Processing

A post-processing script that takes the SBC's synced .csv file and draws the flight path on an interactive map was created. The script is shown in Appendix A.5. When a new radar capture starts, a marker is placed on the interactive flight map at the location. The script erases the first row of data, as the longitude and latitude start at invalid value 0. This is because the

flight controller initially outputs default values before beginning data recording. Coordinates are already in decimal degrees from the SBC program, so they are plotted directly. The script was run on a desktop personal computer (PC), not the SBC. Data visualisation was done after capturing, so using a PC was appropriate. SDR data was not processed in this stage. An example output is shown in Figure 5.5.

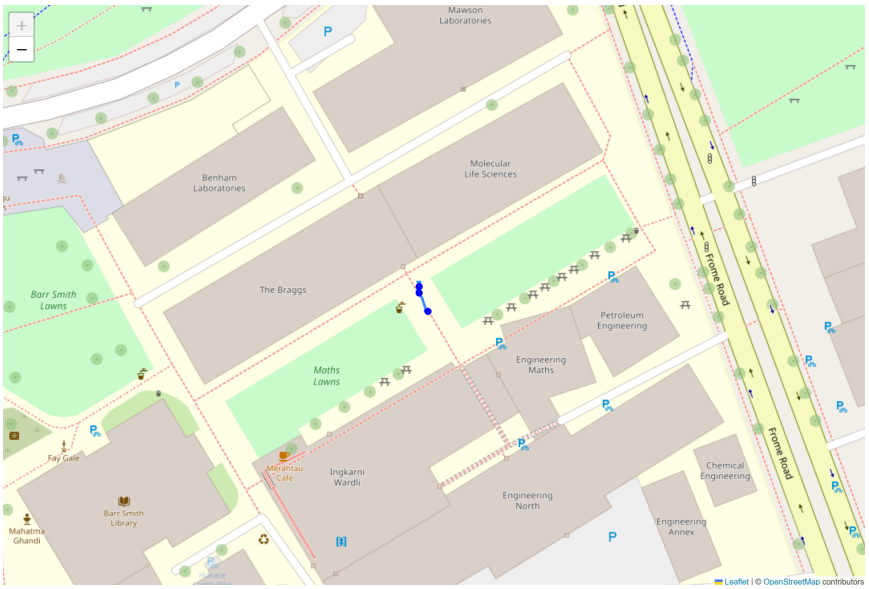


Figure 5.5: Flight path and radar capture locations plotted in the data visualisation script.

The blue line is a point-to-point line between GPS data points, and the markers indicate points where radar data was captured. The markers also contain information about each radar capture (row, capture count), and there is one marker per radar capture.

5.3 Results and Discussion

To verify the control system, three tests were conducted, evaluating the key requirements of the system; GPS, radar capture response, and computational overhead. The GPS drift and radar tests were done using existing testing data; the five tests from the ground integration testing. This was due to it being infeasible to carry the drone out for another ground integration test solely for the purpose of obtaining benchmarks on one subsystem. The computational overhead test was conducted on a seperate occasion, as it was possible to do so from indoors.

5.3.1 GPS Drift

Windows of time where the drone was stationary during the ground integration tests were taken as data samples used to measure stationary GPS drift, where "stationary" was interpreted as having a groundspeed of 0.1m/s. The mean latitude and longitude of the stationary windows are shown in Table 5.2.

Table 5.2: Mean position of each stationary window

Window	Duration (s)	Mean latitude (°)	Mean longitude (°)
Log 1-A	72	−34.9183751	138.6055408
Log 2-A	33	−34.9183822	138.6055543
Log 3-A	16	−34.9183846	138.6055478
Log 3-B	125	−34.9183355	138.6055030

The standard deviation of the distance from the mean, and the maximum distance of the mean are shown in Table 5.3.

Table 5.3: Horizontal GPS drift and satellites visible for each stationary window

Window	RMS distance from mean (m)	Max. distance from mean (m)	Satellites visible, mean (min–max
Log 1-A	0.61	1.14	30.2 (27–32)
Log 2-A	0.05	0.11	30.5 (28–32)
Log 3-A	0.07	0.17	31.8 (31–32)
Log 3-B	0.34	0.65	31.0 (23–32)

The horizontal standard deviation ranged from 0.05 to 0.61 m, with the worst case maximum distance from the mean being 1.14 m.

Log 2-A and Log 3-A had highest stability, with standard deviations of 0.05 m and 0.07 m respectively. The largest drift was shown by Log 1-A, which had standard deviation of 0.61 m. The position moved steadily away from the starting point, reaching 1.82 m after 72 s, despite groundspeed remaining below 0.03 m/s. Log 3 B was in between, with a standard deviation of 0.34 m. There is a large dip in the satellite count for log 3-B. It can be seen that Log 1-A has the same approximate satellite count as Log 2-A, yet has a standard deviation of 0.61 m against 0.05 m. From this, it is clear that the difference in drift does not correspond (in this case) to the satellite count alone.

The two shortest windows had the lowest GPS drift, so it is evident that drift may grow with window length. However, drift is not necissarily directly proportional to duration, as the highest drift was recorded by Log 1-A, which has a shorter duration than Log 3-B.

5.3.2 RC Switch Trigger Latency

An average of how long it took for the radar to capture between activating the trigger and starting the capture was taken for all five logs from the ground integration test. Between the first four tests and the last one, the settling time of the radar before capture was changed, which affected the latency. As such in the results, the first four tests the mean latency is subtracted by 4 s to offset the settling time. The results are shown in Table 5.4.

Table 5.4: Trigger-to-capture latency after removing the 4 s settle-delay from logs 1–4

Group	Events	Mean latency (s)	Std. dev. (ms)
Logs 1–4	10	2.0084	0.41
Log 5	3	2.0030	0.24
All	13	2.0071	2.4

The mean latency across all tests is approximately 2s. The low standard deviations of 0.24 ms and 0.41 ms within each configuration shows that the trigger path is stable. The higher standard deviation when all the data is combined is a reflection of the two sets being mixed dispite being in differnt configurations. The program logged data at 1 Hz, so the switch could have flipped up to 1 ssecond extra before the logged row, so the true latency is thus between approximately 2 and 3 seconds, and the numbers shown in Table 5.4 is a lower bound.

5.3.3 Computational Overhead

To address the tight computational requirements of the airborne system, a computational overhead test was conducted. Three seperate tests were run. The first test was recorded while the system was sitting idle and not running the program. The second test was recorded during program runtime and no radar captures were made. The third test was recorded during program runtime while continously capturing radar data. The first test provides a baseline for the system usage, the second a baseline for the program, and the third the behaviour of the system during operation. The results for the three tests are shown in Table 5.5

Table 5.5: Raspberry Pi resource usage for each test condition

Condition	Samples	Duration (s)	System CPU (%)	Script CPU (%)	RAM used (%)	Temp. (°C)
Baseline (script off)	313	313.5	0.07	–	3.60	48.17
Script running	163	163.3	0.12	0.16	3.77	47.00
Script running, capturing	171	171.3	0.30	0.17	3.85	47.57

Each test made a continual recording of the percentage CPU used by the system, percentage CPU used by the program, percentage RAM used by the system, and the temperature of the system. The system CPU usage increases steadily across the three tests, but remains at less than 1% usage. The script CPU usage is higher than the system CPU usage on one occasion. This is not an error, and is due to the system CPU usage being averaged over all four CPU cores, while the script is measured against one core. The RAM usage increases from 3.6% to 3.85%, and the average temperature remains approximately constant. From this it can be concluded that the program utilises a substantially small amount of computational resources, and so downsizing the SBC in a future iteration is a feasible option, saving mass of the system.

5.4 Summary

Objective 4 aimed at implementing the full control cycle for the airborne SDR system. Where a switch from the transmitter propagates though the system and activates a radar capture. Objective 4 also aimed at synchronising the radar and telemetry data sets, allowing the radar capture to be georeferenced. A Raspberry Pi 5 was chosen due to the SDR USB 2.0 requirement and for room for future onboard processing. The interface was designed at three phases, and ultimately ended with a MAVLink connection over USB. The MAVLink version of the program simplified the wiring and removed the safety switch dependency present in the previous two versions. Logging of telemetry and radar state was done by one Python 3 program and data was logged to one .csv file per program run. The activation logic was a latched trigger which runs a flush, settle, capture and cooldown sequence of radar capture. Capture locations of radar data were plotted on a separate PC program on an HTML map for visualisation. The stationary GPS drift of the system had a standard deviation of 0.05 to 0.61 m and a worst case of 1.14 m

from the mean. The trigger to capture latency averaged 2.007 seconds across 13 documented capture events, however due to the refresh cycle of the system the true capture latency is 2-3 seconds (which can be adjusted). The computational overhead of the program was negligible, the CPU usage remained below 1% and the RAM usage rose from 3.6% to 3.85%, which suggests feasibility for a smaller SBC system. The limitations are the settle-delay setting error in the first four logs, and the reused ground data tests. Future work involves either downscaling the hardware or creating a more sophisticated program that allows the full use of the computational power.

Statement of AI Use

Spelling and Grammar

The project team used ChatGPT (https://chat.openai.com/) and Claude (https://claude.ai/) for this Progress Report to check spelling and grammar in selected text drafted by the team. The use of AI tool was limited to identifying spelling and grammar errors. The tools were not used to draft new report sections, produce summaries or paraphrases, restructure the report, interpret results, draw conclusions. The project team reviewed all suggestions before making any changes and retained copies of the original text submitted to the AI tools.

The following prompt was used in May 2026:

> Please review the following paragraph and identify spelling and grammar errors only. Do not rewrite the text, change the meaning, improve the flow, shorten sentences, add content, or make style or substantive editing suggestions.

Initials of group members who used this support: HL, LT, KM, AS.

Research Technical Support

The project team used Claude Code to analyse the legacy MATLAB driver, implement radar commands, and produce code documentation. All code and documentation generated by AI underwent human review and approval. AI use in this form was approved verbally by Dr Brian Ng. The following prompt was used in April 2026:

> Document all functions in the following file using Doxygen-style comments:
>
> `C:\Users\eliza\SDR_main\SDR_main\linux\src\UsbDevice.cpp`
>
> Documentation requirements:
>
> - Add a Doxygen comment block above every function definition.
> - Include:
>   - `@brief`
>   - `@param` for all parameters
>   - `@return` where applicable
>   - `@note` or `@warning` if relevant
> - Keep descriptions concise but technically clear.

Initials of group members who used this support: HL

6 System Design

This section provides a high level overview of the system that was developed to complete the project, and the problems that were addressed. Section 6.1 addresses the key constraints of the project, deriving the four technical objectives from the problems. Section 6.2 provides an overview of the methodology that was undertaken in the completion of the four technical objectives.

6.1 Problem Definitions

The radar in possession was a Luswave PUP K-band SDR, controlled from a laptop over USB. Integrating it with a uncrewed aerial vehicle (UAV) brought constraints on mass, size, power, mounting and autonomous operation. The five primary problems that had to be addressed were as follows; the ability to control and operate the radar while it is mounted, a secure mount with an unobstructed downward field of view, a power distribution system enabling power to the radar and any peripheral components, the ability to synchronise drone telemetry data sets with radar captures, and the project and operational constraints such as the requirement for the system to be commercial off-the-shelf (COTS), have K-band operation, and the remote-pilot licensing and university rules. Corresponding to the problems are technical objectives aimed at addressing them, the completion of which culminated with the completion of the project.

For radar control, there were numerous constraints. The drone does not have an onboard computer. Also, legacy vendor drivers are discontinued and two are incompatible. The MATLAB API is also poorly documented, which presented difficulty. The final key constraint was that the SDR required USB 2.0 for operating. For mechanical mounting, it was critical to have a secure mount for the radar with an unobstructed downward field of view, clear of the airframe and landing gear. The Aurelia X6 drone had a 5 kg payload limit [**?**], which was another constraint. For the power supply, it was required for it to be independent of the flight power system for modularity. The power supply was required to step 22.2 V down to 6 V to supply the radar, and 5 V to supply the SBC. For the data synchornisation, the radar and telemetry were recorded by seperate devices on seperate clocks, and each data set had to be synchronised. Each radar capture also required both position and attitude for georeferencing. Finally, the activation signal for radar capturing was required to be from the pilot's transmitter without a ground operator, to allow the system to be fully operated by one individual. For system-level and operational constraints, components were predominantly COTS and replicable, so that a future research team can rebuild the system, addressing the literature gap of non-repeatability. The band of the radar did not impose any further constraints, as K-band operation is commercial. Remote-pilot licensing and university rules on drone flights ruled out flight testing as a viable option. The five problems are shown in Table 6.1, in addition to the corresponding constraints and the objective that addresses each.

Table 6.1: problems, constraints and the objective addressing each

Problem	Definition and Constraints	Addressed by
Radar control	No onboard host computer; legacy vendor drivers discontinued and two incompatible, poorly documented MATLAB APIs; SDR requires USB 2.0 high-speed (480 Mb/s)	Objective 1
Mechanical mounting	Secure mount with an unobstructed downward field of view, clear of the airframe and landing gear; 5 kg payload budget	Objective 2
Power supply	Independent of the flight power system; 22.2 V auxiliary supply stepped down to 6 V (SDR) and 5 V (SBC) within tolerance;	Objective 3
Data synchronisation	Radar and telemetry recorded by separate devices on separate clocks; each capture requires position and attitude for georeferencing; pilot must trigger captures from the transmitter without a ground operator	Objective 4
Operational	Predominantly COTS and replicable; K-band operation; remote-pilot licensing and university rules restricted validation to ground testing	All

6.2 Plan for Work

This section provides an overview of the methodology that each objective underwent during development.

6.2.1 Objective 1: Develop SDR-to-SBC Interface for Radar Control and Data Acquisition

The methodology for interfacing the SDR with the SBC followed a structured software engineering lifecycle. It included evaluation of the legacy interface, software implementation, unit testing, and system-level validation. All project source code was authored, reviewed, tested, and version-controlled by the project team.

Research and Evaluation of Existing Interfaces

The first phase reviewed the software interfaces provided by the radar manufacturer [**?**]. This included analysing the legacy MATLAB-based drivers to understand the communication logic and determine whether existing libraries could be reused. The outcome determined that the legacy drivers could be adapted for the target hardware. These findings also defined the technology stack for implementation.

Software Implementation

Following the research phase, an API was developed and implemented. Development was divided into two stages: establishing the communication protocol and implementing the radar command structure. First, the USB firmware protocol was implemented to manage the data link between

the radar and the SBC. The radar control commands, which configure transmit and receive parameters, were then mapped and integrated into the API. To support deployment on the target platform, the API was built as a static library compatible with Ubuntu 22.04. Due to the large variety of radar commands involved and the limited project timeframe, Claude Code was used to assist in implementing the different commands once the command protocol structure had been designed. A human-in-the-loop approach was adopted, where all outputs generated by Claude Code were reviewed and verified to ensure correctness.

At the end of this stage, the USB communication protocol underwent preliminary testing on a standard Linux computer connected to the radar. The radar control commands also received basic functional verification. This testing removed compile-time errors and performed initial validation checks, such as confirming that captured data volume was consistent with the configured sampling rate. Rigorous data-integrity checks and command-level validation were completed during unit testing.

Unit Testing

Initial unit testing was conducted in a standard Linux desktop environment to isolate variables associated with embedded hardware. The radar receiver channel was connected to a known signal generated by a signal generator, while the transmitter channel was connected to a frequency analyser to observe the transmitted output. Individual radar configuration parameters, such as sweep time, sampling frequency, and active channels, were varied one at a time. Each command was then validated by inspecting the API output and laboratory equipment measurements to verify the transmitted and received radar waveforms, ensuring that the radar parameters updated correctly.

Ground Integration and Testing

Once the API passed desktop unit testing, the static library and associated executables were ported to the target SBC, a Raspberry Pi. The SBC, software interface, and power system were then integrated onto a moving ground platform. Ground testing included a controlled motion Doppler test, where the radar was mounted on a ground vehicle moving at a constant velocity. In this scenario, the measured Doppler shift was compared against the expected velocity. Testing verified that valid radar data was generated and stored in the designated Raspberry Pi memory locations under operational scenarios. Additionally, stress-test scenarios, such as extended-duration data capture, were conducted to evaluate system stability. Objective 1 was complete once the system passed the ground integration test.

6.2.2 Objective 2: Design and Integrate a Mechanical Mounting and Protection System for the Payload

Objective 2 addressed the mechanical integration of the radar payload with the selected UAV platform. The method followed a staged process of platform assessment, payload measurement, computer-aided design (CAD) modelling, prototype manufacture, installation, and ground-based verification. This ensured that the radar was securely mounted while flight stability was maintained, the electronics were protected, and an unobstructed field of view was preserved.

Platform and Payload Assessment

The first stage assessed the Aurelia X6 to identify suitable payload mounting locations. The underside of the drone was prioritised because it provided the clearest downward-facing radar view and reduced obstruction from the airframe. The payload bay, underside clearance, landing

gear, wiring paths, and existing mounting points were inspected to determine the most suitable integration approach.

The payload was divided into external and internal subsystems. The external subsystem included the radar antenna, gimbal, and custom mounting brackets, while the internal subsystem included the single-board computer, power components, wiring, connectors, and supporting electronics within the payload bay. This arrangement positioned the radar antenna for measurement performance while keeping the supporting electronics protected inside the drone structure.

Mechanical and geometric requirements were determined through direct measurement and CAD inspection. Key dimensions, component masses, mounting points, cable routes, clearance allowances, and the expected centre of mass were recorded to define the mounting geometry and confirm compatibility with the UAV payload capacity [**?**].

Mounting System Design

A CAD model of the integrated payload system was developed to assess the mechanical layout before manufacture. The model included the underside of the drone, payload bay, radar antenna, gimbal, internal electronics, brackets, and clearance regions. It was used to check for spatial conflicts, fastener access, cable routing, and radar field-of-view obstruction.

The radar antenna was to be mounted beneath the airframe using a gimbal-supported arrangement. The gimbal improved antenna stability, reduced the effect of drone attitude changes on the radar pointing direction, and maintained clearance from the drone body and landing gear. Custom brackets were designed to interface the radar antenna with the gimbal. These components were intended for three dimensional (3D) printing to support rapid manufacture and design iteration. The bracket geometry was based on measured radar and gimbal dimensions, including screw-hole locations, cable exit directions, required offsets, and antenna orientation. The design aimed to minimise mass while maintaining sufficient stiffness and fastener support. The internal electronics layout was also developed in CAD. Components were positioned on standoffs inside the payload bay to provide secure mounting and maintain access to connectors, storage media, and wiring. Cable routes were planned to avoid strain, prevent loose wiring, and ensure that cables did not interfere with the gimbal or radar field of view.

During testing, it was found that this initial enclosure provided insufficient mechanical isolation and support for the antenna assembly. A dedicated enclosure beneath the battery box was found to provide a more practical arrangement. The enclosure incorporated mounting flanges that aligned with the available undertide mounting locations on the UAV. The gimbal was also reassessed and replaced with a more rigid mounting arrangement, as the gimbal increased unecessary mechanical complexity of the system.

Prototype, Manufacture and Installation

Once the CAD model satisfied the initial spatial, mass, and thermal requirements, prototype components were manufactured using 3D printing. Additive manufacturing was selected because it allowed custom geometry to be produced quickly and revised after fit-up testing.

The internal payload layout was then assembled by mounting the electronics on standoffs. Wiring paths were checked so cables could be restrained and connectors remained accessible. Once the internal and external subsystems were confirmed, the payload was installed onto the Aurelia X6 by securing the enclosure, attaching the radar mount, installing the antenna, and restraining wiring with appropriate strain relief.

Mechanical Verification

The completed mounting system was verified through ground-based checks. A static inspection confirmed that the brackets, fasteners, standoffs, electronics, and wiring were secure, and that the payload did not interfere with the landing gear, propellers, airframe, or access panels.

Structural stability was checked by gently loading the mounted payload in the vertical and lateral directions to identify looseness, excessive flexure, cracking, or fastener movement. Any movement or deformation would have required reinforcement or redesign before flight testing.

The final check involved powered ground testing. The electronics were operated while installed on the drone to confirm that the payload remained secure and that wiring did not move into the gimbal path or radar view. Where permitted, a controlled ground-run test was used to assess vibration effects. The assembly was placed on a moving platform where the vibrations were controlled to accurately simulate flight dynamics. Mechanical integration was considered complete once the payload was secure, the radar field of view was clear, and all electronics and wiring were restrained.

6.2.3 Objective 3: Design, Build, and Verify the Onboard Power System

Objective 3 addressed the design, construction, and verification of an onboard power system capable of supplying the SDR, SBC, gimbal, and supporting electronics during flight operations. The method followed four stages: design, prototyping and controlled testing, construction, and integration testing. This ensured that the power system met electrical requirements, was verified before each stage progressed, and could operate reliably.

Design

The first stage identified the electrical and power requirements of all loads. Voltage, current, and power requirements for the SDR, SBC, and gimbal were extracted from their technical datasheets, with typical and maximum values recorded. The minimum energy required for the 30-minute flight duration was then calculated for each load and for the full system [**?**].

Once the requirements were established, a high-level design addressed energy capacity, output voltage and current, safety, and compatibility. For each requirement category, the necessary components and specifications were determined. The output of this process was a single-line diagram (SLD) defining the complete system architecture. Following this, components that satisfied the system requirements were selected and procured.

Prototyping and Controlled Testing

Before physical assembly, the power system design was verified through circuit simulation. The specifications of selected components were entered into a simulation model, and key outputs, including total load power, system efficiency, source current draw, and voltage drop, were checked against the design requirements.

Unit testing was then conducted for each component. Each component was tested at its nominal input voltage, with the load increased incrementally until the required operating conditions were reached. Voltage, current, and temperature were recorded at each step to confirm correct operation and identify thermal concerns before system-level testing [**?**].

A staged energisation plan was then developed for complete-system bench testing. The plan defined the sequence in which loads were connected, with voltage, current, power, and temperature recorded at each stage. This approach confirmed performance incrementally and helped

to identify and isolate faults. The energisation plan was then followed across all loads to verify that the complete system met power and safety requirements.

Construction

During prototyping and controlled testing, the power system used temporary electrical connections so components could be adjusted or replaced if issues were identified. Once the design was verified through the full energisation plan, temporary connections were replaced with soldered joints. Each connection was insulated to prevent short circuits and improve mechanical robustness. The completed assembly was inspected before integration testing.

Integration Testing

The constructed power system was verified through ground-based and airborne tests. A ground test was first conducted with the power system installed on the drone to verify that it could safely and reliably support all subsystems for the required operational duration. Electrical parameters were recorded to confirm compliance with the design requirements. Objective 3 was complete once the power system reliably supplied all loads during ground testing [**?**].

6.2.4 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements

Objective 4 addressed the synchronised logging of radar measurements and UAV flight telemetry to ensure that time-aligned datasets are collected for research purposes. The method followed a staged process covering SBC selection, operating-system setup, activation-signal testing, telemetry and logging configuration, dataset alignment and system testing. This ensured that radar measurements could be interpreted against the corresponding UAV position, attitude, altitude, and time history.

SBC Selection and System Setup

The first stage involved selecting a suitable SBC for onboard data logging and control. The SBC was chosen based on the SDR specifications, including evaluation of the required input/output interfaces and the data throughput demands of the SDR [**?**]. In addition, the selection considered onboard processing capability, storage capacity, physical size, and power consumption. The selected SBC had to be able to receive a flight-controller activation signal, initiate radar logging, and store SDR data locally.

After selection, an operating system was installed and configured for onboard use [**?**]. This included preparing the boot media and enabling the required interfaces. The SBC was configured to detect an external trigger from a designated input pin. A bench test was then conducted using a signal generator to verify that the SBC could detect the trigger and execute a testing script reliably.

Telemetry and SDR Data Logging

This stage ensured that both telemetry and SDR logging were initiated correctly when the flight controller received a command from the ground station.The flight controller was configured to record telemetry required for post-processing, including time, position, altitude, velocity, attitude, heading, and other relevant flight-state information where available. The telemetry log format, sampling rate, file location, and export method were documented so the process could be repeated during future tests. Telemetry logging was validated by recording data while the drone was stationary on the ground. The recorded data were inspected to confirm that the

required parameters were present, the data were recorded at an acceptable sampling rate.

In parallel, a program to interface with the flight controller and receive the pilot activation signal was developed. The program was first developed to interface with one of the servo channel outputs of the flight controller, however it was found during subsequent development that interfacing with USB over the MAVLink protocol was more practical. This was due to MAVLink requiring fewer wires (one as opposed to two) and being able to transmit both the activation signal and all telemetry from the drone. All telemetry being transmitted allowed data from both the radar and the drone to be synchronised on the SBC.

Post Processing and System Verification

A post-processing method was developed to visualise the now synchronised radar and telemetry data sets. The method plotted the GPS data points on an HTML map and marked the points where a radar capture was taken. The MAVLink program was verified across three metrics; GPS drift, activation signal delay, and computational overhead. The three tests encapsulated the functional requirements of the system (data, activation, and low resource consumption), and the completion of these tests marked the completion of objective 4.

A Objective 4: Supporting Material

A.1 SSH Setup

Listing A.1: The commands used to setup SSH connection between the Raspberry Pi and a remote computer.

```
sudo apt install openssh-server
sudo systemctl enable ssh
sudo ufw allow ssh
```

A.2 GPIO Packages Install

Listing A.2: Bash command used to install python GPIO library.

```
sudo apt install python3-lgpio gpiod
```

A.3 Virtual Environment and `rpi-lgpio` Install

Listing A.3: Commands used to create the python virtual environment and install `rpi-lgpio`.

```
wget http://abyz.me.uk/lg/lg.zip
unzip lg.zip
cd lg
make
sudo make install
python3 -m venv gpio-env
source gpio-env/bin/activate
pip install rpi-lgpio
```

A.4 GPIO Read Script

Listing A.4: Python script used to run on the Raspberry Pi to monitor one of the GPIO pins.

```
#!/usr/bin/env python3
import RPi.GPIO as GPIO # Uses rpi-lgpio backend automatically
from time import sleep
import signal

# Setup
GPIO_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(GPIO_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

print("Sensing 3.3V High...")

try:
    while True:
        if GPIO.input(GPIO_PIN):
            print("Pin High")
            sleep(1)
except KeyboardInterrupt:
    print("Stopping...")
```

```
finally:
    GPIO.cleanup()
pause()
```

A.5 Telemetry Data Visualisation Script

Listing A.5: Python script used to plot and visualise syncrhonised radar and telemetry data

```
import folium
import pandas as pd
import os

# parameters
filename = "flight_log_5.csv"


# Code
df = pd.read_csv(filename)

# first lat/lon sample is invalid, drop it and reindex from 0
df = df.iloc[1:].reset_index(drop=True)

output_file = os.path.splitext(filename)[0] + ".html" # -> "flight_log_4.html"
os.remove("synced_data.html") if os.path.exists("synced_data.html") else None

m = folium.Map(location=[df.at[0, "latitude"], df.at[0, "longitude"]], zoom_start=100)

# Create list of [latitude, longitude] points
points = list(zip(df["latitude"], df["longitude"]))

# Draw flight path
folium.PolyLine(points, weight=3).add_to(m)

capture_count = 0
for i, row in df.iterrows():
    tooltip_html = (
        f"<b>#{i}</b><br>"
        f"Latitude: {row['latitude']:.6f}<br>"
        f"Longitude: {row['longitude']:.6f}<br>"
        f"Capture Count: {row['radar_capture_count']}<br>"
    )

    if row["radar_capture_count"] == capture_count + 1:
        folium.CircleMarker(
            location=[row["latitude"], row["longitude"]],
            radius=3,
            color="blue",
            fill=True,
            fill_opacity=0.8,
            tooltip=tooltip_html,
        ).add_to(m)
        capture_count = 1 + capture_count
# Save map
```

```
46 m.save(output_file)
```